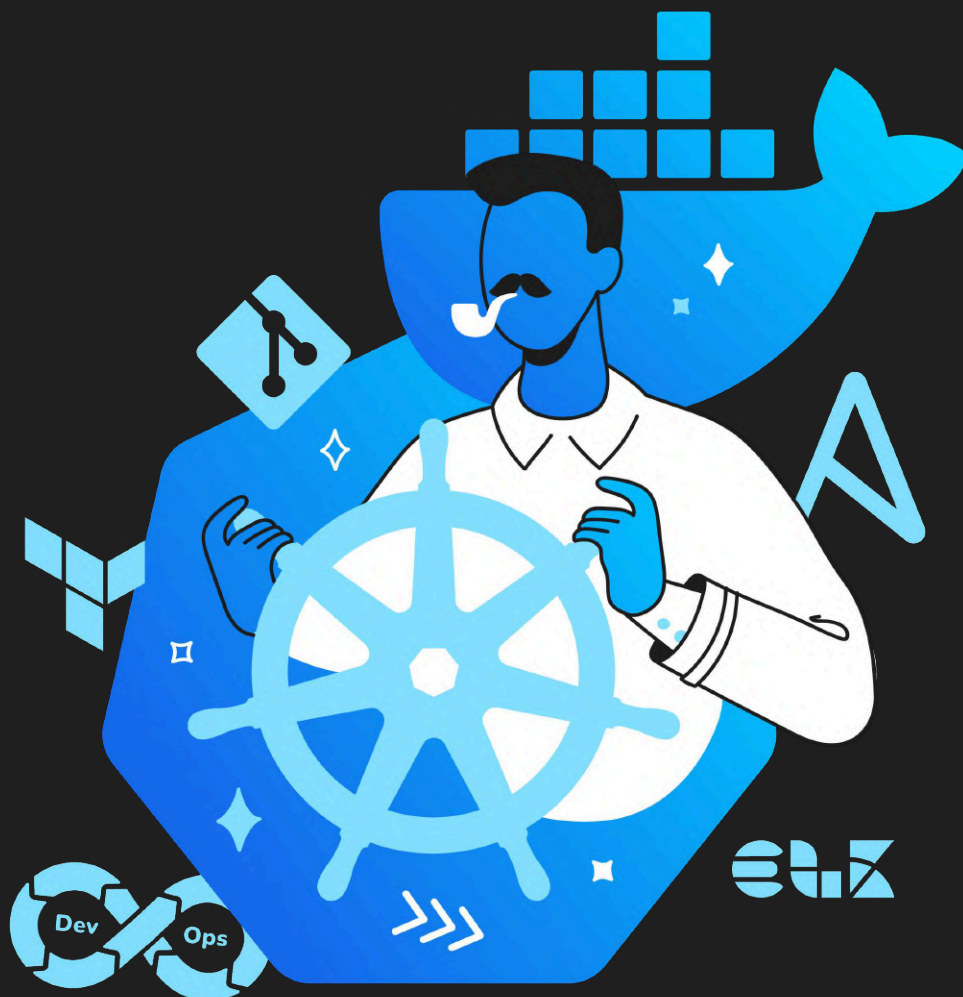


КАК СТАТЬ DEVOPS. ПОДРОБНЫЙ ГАЙД И РОАДМАП





СОДЕРЖАНИЕ

Рoadmap	2
Кто такой DevOps и почему каждый хочет им стать	3
Языки программирования	4
Операционные системы и работа в терминале	6
Контроль версий Git	7
Контейнеризация	8
Сети и безопасность	9
Инфраструктура как код (IaC)	10
CI/CD	11
Мониторинг и логирование	12
Облачные провайдеры	13
Не переставайте учиться	14

- ✓ Персональная рекомендация / личное мнение
- ✓ Альтернативный вариант, как опция первому
- ✓ Можно учить в любом порядке
- Не советую

DevOps





Кто такой DevOps и почему каждый хочет им стать

DevOps-инженер — это специалист на стыке разработки и эксплуатации, ответственный за стабильное и быстрое развертывание приложений. В его задачи входит автоматизация, настройка CI/CD, мониторинг, и обеспечение непрерывности процессов.

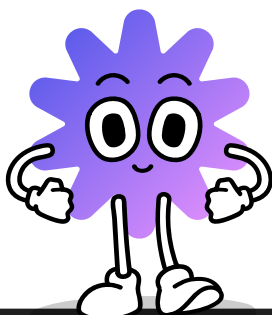
По [данным](#) HeadHunter, спрос на DevOps-специалистов остается высоким, особенно, если речь идет о специалистах с опытом от 3 лет. Медианная зарплата джуна составляет 85 тысяч рублей в месяц. Подробнее о профессии и её особенностях мы писали [в базе знаний Merion](#).

Смотрите наш [обзор](#) о том, что такое DevOps и что нужно знать DevOps-инженеру.

А можно ли стать DevOps-инженером с нуля?

Стать DevOps-инженером с нуля вполне реально, но этот путь непростой. В чистом виде навыкам DevOps не обучают в университетах. Часто специалисты приходят сюда из смежных сфер: системного администрирования, разработки, тестирования. Многие DevOps — самоучки. Базовые знания можно получить через онлайн-курсы, изучение технического руководства и работу с опытными коллегами. Однако, чтобы стать полноценным DevOps, обычно требуется несколько лет практического опыта. От DevOps-инженера ожидается, что он сможет принимать технически сложные решения, а для этого нужно набить руку и пройти через собственные ошибки.

Мы систематизировали для вас дорожную карту и собрали полезные ссылки на обучающие материалы.





ШАГ 1. ЯЗЫКИ ПРОГРАММИРОВАНИЯ

Неважно, какой язык вы выберете, но важно освоить хотя бы один.

Он пригодится для написания скриптов автоматизации. Python подойдет для автоматизации и написания скриптов, Go или JavaScript (Node.js) для работы с инфраструктурными задачами.

Python

Python — это простой и понятный язык программирования. Он поддерживает множество парадигм программирования, включая процедурное, объектно-ориентированное и функциональное программирование.

У Python есть большая встроенная библиотека и множество готовых пакетов. Это делает его удобным для веб-разработки, анализа данных, искусственного интеллекта и научных расчётов. Понятный синтаксис и динамическая типизация позволяют быстро создавать и тестировать код.

Кроссплатформенность: Python хорошо работает на разных платформах, поэтому программы легко переносить с одного устройства на другое. Благодаря активному сообществу и постоянному развитию Python стал одним из самых популярных языков, который используют для любых задач — от простых скриптов до сложных open-source проектов.

[Разобраться в Python можно тут](#)

[Здесь мы рассказываем про Python и почему вам захочется его изучить](#)

[А еще подборка статей про Python \(их у нас 150\)](#)

Go

Go или Golang — это язык программирования, который создала компания Google. Он отличается простотой и высокой производительностью. Go быстро компилируется, автоматически управляет памятью и имеет мощную стандартную библиотеку. Его синтаксис понятен, упрощает чтение и поддержку кода.



В Go есть инструменты для многозадачной работы — goroutines и каналы. Он хорошо подходит для создания системных программ, сетевых сервисов и облачных приложений. Go экономно использует память и быстро работает, поэтому его часто выбирают для серверных приложений и микросервисов. Из-за простоты и скорости Go стал популярен в DevOps и облачных технологиях.

JavaScript / Node.js

Следующим в нашем списке идет JavaScript. В DevOps он пригодится для написания скриптов, управления инфраструктурой и создания удобных инструментов. А в связке с Node.js он позволяет настраивать API, создавать серверные скрипты и оптимизировать развертывание приложений.

JavaScript — это язык программирования, который используется в основном для создания веб-сайтов. Он позволяет добавлять интерактивные и динамичные элементы на страницы, что делает их более удобными для пользователей. JavaScript работает вместе с HTML и CSS.

Язык поддерживает разные стили программирования, такие как объектно-ориентированное и функциональное. С появлением Node.js его также начали использовать для серверной разработки.

JavaScript имеет динамическую типизацию, функции первого класса, множество библиотек и фреймворков (React, Angular и Vue.js). Это делает его одним из самых популярных языков программирования.

[Подборка статей по JavaScript](#)

Про Node.js мы кратко рассказываем [тут](#) и [тут](#)



ШАГ 2. ОПЕРАЦИОННЫЕ СИСТЕМЫ И РАБОТА В ТЕРМИНАЛЕ

Важный шаг для будущего DevOps-инженера — изучение ОС. Большинство серверов работают на Linux и Unix системах. Linux — это основа, на которой строятся многие процессы в DevOps. Особенно популярны дистрибутивы вроде Ubuntu, CentOS, RHEL или Debian.

Углубившись в Linux, важно освоить базовые навыки:

- **Работа с файловой системой:** понимание структуры каталогов, создание и редактирование файлов
- **Управление сервисами:** настройка и запуск через Systemd или аналогичные системы
- **Управление пользователями и правами:** создание пользователей, групп, настройка прав доступа
- **Установка и обновление ПО:** работа с пакетными менеджерами, сборка программ вручную
- **Сетевые настройки:** конфигурация соединений, настройка фаерволов
- **Мониторинг и управление процессами:** использование инструментов для отслеживания нагрузки
- **Автоматизация:** написание Bash-скриптов для выполнения рутинных задач

[Про Linux за 5 минут](#)

[Все, что есть в нашей базе знаний про Linux](#)

Кроме того, терминал должен стать вашим лучшим другом, так как большая часть задач DevOps решается именно здесь.

Освойте основные скрипты: Bash или PowerShell (для Windows-сред), чтобы автоматизировать процессы в операционной системе.



ШАГ 3. КОНТРОЛЬ ВЕРСИЙ GIT

Git — стандарт для контроля версий и незаменимый инструмент для DevOps-инженера. Он пригодится, чтобы понимать проект на всех этапах разработки и эффективно работать над кодом вместе с другими участниками команды.

Почему важно знать Git:

- **Полный контроль над проектом.** Git позволяет отслеживать историю изменений кода и оперативно находить ошибки
- **Работа с ветками:** команды ветвления и слияния помогают организовать параллельную работу над фичами, исправлениями и тестированием
- **Обратимость действий.** Если вы допустили ошибку или нашли проблему с кодом, Git позволяет быстро откатить изменения, восстановить рабочую версию и минимизировать риски
- **Совместная работа.** Git помогает эффективно работать в команде, где разные разработчики вносят изменения в одну систему

Основные команды Git:

- **git init** — создание нового репозитория
- **git clone** — клонирование существующего репозитория
- **git add** — добавление изменений в индекс
- **git commit** — сохранение изменений в истории
- **git branch** — работа с ветками
- **git merge** — объединение веток
- **git push** и **git pull** — синхронизация локальных изменений с удаленным репозиторием

[Тут наш бесплатный курс по Git](#)

[И все о Git и GitHub для новичков](#)





ШАГ 4. КОНТЕЙНЕРИЗАЦИЯ

Контейнеризация — это технология, которая позволяет упаковать приложение вместе со всеми его зависимостями в единый изолированный контейнер. В контейнер помещается всё, что нужно приложению для работы: код, библиотеки, конфигурационные файлы и зависимости. Это делает приложение независимым от операционной системы или среды, где оно запускается.

Зачем девопсу уметь собирать образы контейнеров?

- **Унификация среды:** приложение будет работать одинаково на любом сервере, будь то локальная машина или облако
- **Простота управления:** контейнеры легко запускать, останавливать, обновлять и переносить между серверами
- **Масштабирование:** с контейнерами легче распределять нагрузку, увеличить или уменьшить количество экземпляров приложения
- **Изоляция:** каждый контейнер изолирован от других, а это повышает безопасность и стабильность системы

Чтобы быстро упаковать приложение и все его зависимости в контейнер придется освоить **Docker**. Docker — это популярная платформа для создания, управления и развертывания контейнеров.

Когда контейнеров становится много и они требуют автоматизации, нужен **Kubernetes**. Это система оркестрации контейнеров, которая помогает управлять развертыванием, масштабированием и работой приложений в контейнерах. Kubernetes помогает связать их между собой. Например, вы можете настроить, чтобы базы данных, API и фронтенд приложения работали в одной системе.

[Подробнее, как разобраться с Docker](#)

[Полезный вебинар про то, как запустить свой первый контейнер в Docker](#)

[В чем разница между Docker и Kubernetes](#)

[Что такое Kubernetes](#)



ШАГ 5. СЕТИ И БЕЗОПАСНОСТЬ

Основная задача DevOps — обеспечить надежную и безопасную инфраструктуру для приложений. Это невозможно без понимания того, как устроены сетевые технологии.

DevOps работает с **сетевыми протоколами**, которые обеспечивают обмен данными между системами. Важно понимать их работу и настройки:

- **HTTP/HTTPS**: передача данных в веб-приложениях; настройка HTTPS с использованием **SSL/TLS** для шифрования
- **SSH**: удаленное управление серверами и контейнерами; защита доступа с использованием ключей
- **FTP/SFTP**: передача файлов; настройка безопасности для предотвращения утечек.
- **SMTP**: отправка почты; защита от спама и настройка шифрования

[Полный курс по сетевым технологиям](#)

[Полезные материалы по сетям](#)



ШАГ 6. ИНФРАСТРУКТУРА КАК КОД (IaC)

Вместо ручной настройки серверов, сетей и других ресурсов, DevOps инженеры используют специальные инструменты и скрипты, чтобы автоматически управлять инфраструктурой. Такой подход называется IaC или **Инфраструктура как код (Infrastructure as Code)**.

Преимущества IaC:

1. Экономит время и снижает вероятность ошибок, так как вместо ручной настройки всё выполняется автоматически по заранее написанным сценариям
2. Созданный код можно многократно использовать для развертывания одинаковой инфраструктуры в разных средах
3. IaC повышает прозрачность и упрощает совместную работу. Так как инфраструктура описана в виде кода, её изменения можно отслеживать с помощью Git
4. Легко масштабировать ресурсы под меняющуюся нагрузку, например, добавить сервер или увеличить мощность базы данных

Для полного цикла автоматизации инфраструктуры и настройки систем DevOps-инженеры используют **Terraform** и **Ansible**.

Terraform автоматизирует создание серверов, сетей, баз данных, балансировщиков нагрузки и других компонентов. Это упрощает развертывание сложных инфраструктур и их масштабирование.

[Рассказываем про Terraform](#)

Следующий инструмент — Ansible — нужен для управления конфигурацией, оркестрации и автоматизации задач. Он безагентный (работает через SSH) и удобен для настройки серверов, развертывания приложений и управления конфигурациями. С его помощью можно автоматизировать рутинные задачи, управлять контейнерами и приложениями, а также наладить CI/CD.

[Смотрите, что такое Ansible](#)



ШАГ 7. CI/CD

Непрерывная интеграция и доставка (CI/CD) позволяют разработчикам и командам работать быстрее и надежнее.

Непрерывная интеграция (Continuous Integration, CI) — процесс, при котором изменения в коде автоматически проверяются и объединяются в основной проект. Например, при каждом внесении изменений система запускает тесты, проверяет совместимость нового кода с существующим, чтобы избежать ошибок.

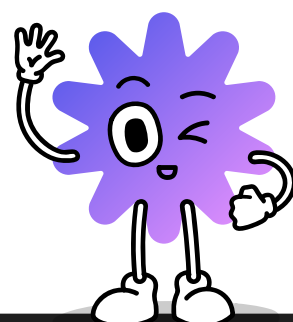
Непрерывная доставка (Continuous Delivery, CD) — это автоматизация процесса подготовки приложения к развертыванию. Программа всегда готова к выпуску, и всего одной командой её можно развернуть в любое время.

Для реализации CI/CD DevOps-инженеру нужно знать хотя бы один инструмент, например, [Jenkins](#) или GitLab CI/CD. Освоив один, легко перейти на другие. Это поможет быстрее добавлять новые функции в приложения, сохраняя их стабильность.

[Здесь мы писали про основы CI/CD](#)

[И в видео-формате за 5 минут](#)

[Про GitHub Actions](#)





ШАГ 8. МОНИТОРИНГ И ЛОГИРОВАНИЕ

Эти процессы помогают следить за состоянием системы, находить проблемы и обеспечивать её стабильную работу. Без мониторинга невозможно развитие приложения. Вот основные задачи, которые выполняет девопс:

1. Сбор метрик. DevOps настраивает инструменты, чтобы автоматически собирать данные о состоянии серверов, приложений и сетей
2. Визуализация данных и анализ производительности системы
3. Настройка уведомления о проблемах, чтобы команда могла быстро среагировать
4. Анализ данных и решение проблем
5. Оптимизация инфраструктуры
6. Интеграция мониторинга с CI/CD, чтобы автоматизировать проверку после развертывания новых версий приложений

Инструменты мониторинга:

Prometheus — это система мониторинга с открытым исходным кодом. Он собирает метрики (например, загрузку CPU, использование памяти или скорость отклика сервера) и хранит их в виде временных рядов. Хорошо работает с микросервисами, Docker и Kubernetes.

Grafana — это инструмент для визуализации данных. Он берет метрики из Prometheus (или других источников) и превращает их в информативные графики и дашборды. В нем можно сделать один экран и увидеть, как работает система: нагрузка, скорость, ошибки — всё перед глазами.

Zabbix — ещё один универсальный инструмент для мониторинга. Можно следить за серверами, сетевым оборудованием или приложениями. У Zabbix есть система уведомлений, которая присылает уведомления, если где-то возникает проблема. Например, он пригодится для контроля состояния серверов в офисе или облаке.



ШАГ 9. ОБЛАЧНЫЕ ПРОВАЙДЕРЫ

DevOps-инженер должен понимать принципы работы различных облачных сервисов, так как современные компании часто используют их для гибкости и экономии. Amazon Web Services (AWS) считается лидирующим облачным провайдером облачных технологий. Microsoft Azure — часто используется в корпоративной среде. Для работы с большими данными и искусственным интеллектом хорошо подходит Google Cloud.



ШАГ 10. НЕ ПЕРЕСТАВАЙТЕ УЧИТЬСЯ

DevOps — динамичная область, поэтому придется постоянно прокачивать свои навыки.

Что делать, если я вообще зеленый?

Приходи учиться к нам. Мы запустили [курс](#) по инструментам и методам DevOps. Мы расскажем все от основ автоматизации до настройки CI/CD пайплайнов, контейнеризации с Docker и оркестрации в Kubernetes. Разберем принципы инфраструктуры как кода, научим работать с облачными сервисами, инструментами мониторинга и логирования. Наш курс даст прочную базу, чтобы начать свой путь в DevOps!

Материалы для изучения:

[Что такое Девопс](#)

[Что такое DevOps, что нужно знать и сколько получают](#)

[DevOps — это философия будущего: кто, что и как?](#)

[Курс Девопс с нуля](#)

[Вебинар «Как стать инженером DevOps»](#)

